



Código Limpio (Clean Code)

Robert C. Martín



Introducción

Hoy en día, el objetivo es comercializar rápidamente los productos. Queremos que las fábricas produzcan software a pleno rendimiento.

"...gran parte del trabajo no radica en la fabricación sino en el mantenimiento..."



Introducción

Cada año, se invierten horas y se pierden recursos debido a código mal escrito, ralentizando el desarrollo, disminuyendo la productividad, generando graves fallos e incluso pudiendo acabar con la empresa.



Introducción

El código incorrecto es el motivo del fin de la empresa

¿Por qué el código de calidad se transforma tan rápidamente en código incorrecto?

La culpa es nuestra. No somos profesionales.

Gran parte del trabajo no radica en el desarrollo sino en el mantenimiento del código

Somos cómplices en la programación del proyecto y compartimos gran parte de la responsabilidad de los fallos, en especial si tienen que ver con código incorrecto.



La filosofía Lean 5S

La práctica del software correcto requiere disciplina.

La filosofía 5S incluye estos conceptos:

- **Seiri u organización:** Es fundamental saber dónde están las cosas, Ejemplo: el uso de nombres correctos para métodos, clases, archivos, etc.
- **Seiton o sistematización:** Un sitio para cada cosa y cada cosa en su sitio. Un fragmento de código debe estar donde esperamos encontrarlo; en caso contrario, se debe refactorizar.
- **Seiso o limpieza:** Mantener limpio el lugar de trabajo. Ejemplo: eliminar comentarios que no sirven.
- **Seiketsu o estandarización:** El grupo decide cómo mantener limpio el lugar de trabajo.
- **Shutsuke o disciplina:** Significa ser disciplinado en la aplicación de las prácticas y reflejarlas en el trabajo y aceptar los cambios.

Lean 5S es una herramienta de gestión empresarial que se originó en Japón



Concepto de Código Limpio

Me gusta que mi código sea elegante y eficaz. La **lógica debe ser directa** para evitar errores ocultos, las **dependencias deben ser mínimas** para facilitar el mantenimiento, el **procesamiento de errores completo** y sujeto a una estrategia articulada, y el rendimiento debe ser óptimo para que los usuarios no tiendan a estropear el código con optimizaciones sin sentido. El **código limpio hace bien una cosa**.

Bjarne Stroustrup, inventor de C++ y autor de The C++ Programming Language



Concepto de Código Limpio

El código limpio es **simple y directo**. El código limpio se **lee como un texto bien escrito**. El código limpio no oculta la intención del diseñador, sino que muestra nítidas abstracciones y líneas directas de control.

Grady Booch, autor de Object Oriented Analysis and Design
with Applications



Concepto de Código Limpio

El código limpio se puede **leer y mejorar** por parte de un programador **que no sea su autor original**. Tiene **pruebas de unidad y de aceptación**. Tiene nombres con sentido. Ofrece una y no varias formas de hacer algo. Sus dependencias son mínimas se definen de forma explícita y ofrece una API clara y mínima.

Dave Thomas, fundador de OTI, el padrino de la estrategia Eclipse



Concepto de Código Limpio

El código limpio **siempre parece que ha sido escrito por alguien a quien le importa.**

Michael Feathers, autor de Working Effectively with Legacy Code



¿Qué es el código limpio?

El código limpio es aquel que es **simple, directo y expresivo**. El código limpio debe:

- Ser fácil de leer y entender.
- Evitar repeticiones innecesarias.
- Expresar la intención del desarrollador.
- Ser eficiente y mantenible.



Nombres significativos

El uso de nombres adecuados es esencial para escribir código comprensible. Algunos consejos clave incluyen:

- Usar nombres descriptivos que reflejen el propósito de la variable, función/método o clase.
- Evitar abreviaciones innecesarias.
- Elegir nombres pronunciables y fácilmente recordables.
- Usar nombres que se puedan buscar en el IDE.
- El nombre de una clase no debe ser un verbo, debe ser un sustantivo propio
- Los métodos de una clase en lo posible deben ser verbos (en lo posible en infinitivo)
- Elegir una palabra por concepto (fetch, retrieve y get son equivalentes pero debe usar solamente una en todo el proyecto)
- Usar nombres de dominios de soluciones

Lo más complicado a la hora de elegir un buen nombre es que requiere habilidad descriptiva y principio cultural. Es un problema de formación más que técnico, empresarial o administrativo. Como resultado, mucha gente del sector no aprende a hacerlo bien.



Funciones

Son la primera línea organizativa en cualquier programa.

Las funciones:

- Deben ser pequeñas en cantidad de líneas (**longitud aproximada de 20 líneas**).
- Sólo deben hacer una cosa. Deben hacerlo bien y debe ser lo único que hagan (SRP).
- Deben tener nombres que indiquen claramente su propósito.
- Debem evitar **if anidados** y **switch (match case)**, por lo general incumplen SRP y OCP
- El número ideal de **argumentos** para una **función es 0 a 2**. Siempre que sea posible evitar la presencia de 3 argumentos o más.

SRP: Principio de Responsabilidad Única

OCP: Principio Abierto/Cerrado



Funciones - ¿Cuál se lee mejor?

```
make_circle (x: float, y: float, radius: float) -> Circle:
```

```
make_circle(center: Point, radius: float) -> Circle:
```



Funciones

- Evitar efectos secundarios inesperados.
- Los bloques try/catch confunden la estructura del código.
- La duplicación de código es un problema ya que aumenta el tamaño del código.
- Seguir el principio de "una entrada, una salida". Esta regla implica que sólo debe haber una instrucción return en una función.

Los programadores experimentados piensan en los sistemas como en **historias que contar**, no como en programas que escribir. Las **funciones describen las acciones** que se pueden realizar en el sistema. Las funciones que se escriban **deben encajar en un lenguaje claro y preciso** que le sirva para contar esa historia.



Comentarios

"No comente el código incorrecto, reescríbalo"

Brian W. Kernighan y P. J. Plaugher -
Coautor del libro El lenguaje de programación C y
del desarrollo de Unix.

Buenas prácticas incluyen:

- Usar comentarios para explicar "por qué" se hace algo, no "qué" se hace.
- Los comentarios imprecisos son mucho peor que la ausencia de comentarios
- Evitar comentarios redundantes.
- Preferir código autoexplicativo en lugar de comentarios excesivos.
- Comentarios TODO. TODO son tareas que el programador debería hacer.
- Comentarios obligatorios: Por lo general es buena práctica documentar las funciones.
- Código comentado: no hay peor cosa que el código comentado.



Formato del código

El formato del código afecta la legibilidad y comprensión del mismo:

- **Mantener la coherencia en el estilo de codificación.**
- **Reglas de equipo:** El equipo debe acordar un único estilo de formato.
- Separar lógicamente los bloques de código.
- **Formato vertical:** Se debe tratar de tener menos de 200 líneas de código por archivo/clase
- **Formato horizontal:** Se prefieren líneas menos anchas.
- **Declaraciones de variables:** Deben aparecer en la parte superior de cada función.
- **Variables de instancia / propiedades:** deben declararse en la parte superior de la clase.
- **Funciones dependientes.**



Objetos y estructuras de datos

- El diseño orientado a objetos se basa en la **encapsulación** y la **separación de responsabilidades**.
- **Abstracción de datos:** Mantener la información privada siempre que sea posible. Diseñar objetos de manera que expongan lo mínimo necesario.
- **Antisimetría de datos y objetos:** Los objetos ocultan sus datos tras abstracciones y muestran funciones que operan en dichos datos. La estructura de datos muestra sus datos y carece de funciones con significado.



Objetos - Ley de Demeter

"No hable con desconocidos, sólo con amigos"

La **Ley de Demeter** establece que un objeto debe interactuar solo con sus dependencias directas y evitar acceder a estructuras internas de otros objetos. Esto **reduce el acoplamiento** y hace que el código sea más mantenible.

Un método de una clase solo debe llamar a:

- Métodos de la propia clase.
- Métodos de sus atributos directos.
- Métodos de objetos creados dentro de él.
- Métodos de parámetros recibidos.



Objetos - de Ley Demeter

```
class Direccion:
    def __init__(self, ciudad):
        self.ciudad = ciudad

class Persona:
    def __init__(self, direccion):
        self.direccion = direccion

class Pedido:
    def __init__(self, persona):
        self.persona = persona

    def obtener_ciudad_cliente(self):
        # ❌ Ese tipo de código suele denominarse choque de trenes
        return self.persona.direccion.ciudad
```



Objetos - de Ley Demeter

```
class Direccion:
    def __init__(self, ciudad):
        self.ciudad = ciudad

class Persona:
    def __init__(self, direccion):
        self.direccion = direccion

    def obtener_ciudad(self):
        return self.direccion.ciudad #✅ Encapsula el acceso a la dirección

class Pedido:
    def __init__(self, persona):
        self.persona = persona

    def obtener_ciudad_cliente(self):
        #✅ Obtiene la información sin acceder a la estructura interna
        return self.persona.obtener_ciudad()
```



DTO (Data Transfer Object)

Los DTO son estructuras muy útiles, en especial para comunicarse entre capas de una aplicación, o con otras aplicaciones.

Se debe usar estructuras de datos adecuadas para optimizar el rendimiento



Objetos - Conclusión

Los objetos muestran comportamiento y ocultan datos.

Se deben aplicar principios **SOLID** y **patrones de diseño** para facilitar la inclusión de nuevos tipos de objetos sin necesidad de cambiar los comportamientos existentes.



Manejo de errores

El manejo adecuado de errores mejora la robustez del sistema

- Usar excepciones en lugar de códigos de error.
- Mantener el flujo del código limpio y legible.
- No devolver valores nulos; usar objetos nulos o excepciones.



Pruebas unitarias

Las pruebas unitarias garantizan la funcionalidad del código y facilitan el mantenimiento

Las tres leyes:

- **Primera ley:** No debe crear código de producción hasta que haya creado una prueba de unidad que falle.
- **Segunda ley:** No debe crear más de una prueba de unidad que baste como fallida (que no compile).
- **Tercera ley:** No debe crear más código de producción que el necesario para superar la prueba de fallo actual.



Pruebas unitarias - F.I.R.S.T.

..otras cinco reglas:

- **Rapidez (Fast):** Las reglas deben ser rápidas y ejecutarse de forma rápida.
- **Independencia (Independent):** Las pruebas no deben depender entre ellas.
- **Repetición (Repeatable):** Las pruebas deben poder repetirse en cualquier entorno DEV/PROD.
- **Validación automática (Self-validating):** Las pruebas deben tener un resultado booleano: o aciertan o fallan
- **Puntualidad (Timely):** Las pruebas deben crearse en el momento preciso: antes del código de producción. No desarrolle código de producción que no se pueda probar.

<https://www.paradigmadigital.com/dev/principio-first-aumentar-la-calidad-tests-unitarios/>



Clases

Las clases deben seguir el **principio de responsabilidad única y mantener un tamaño manejable**.

Organización de clases

- De acuerdo a la convención de cada lenguaje de programación se debe seguir un orden.
- **Encapsulación**: Queremos que nuestras variables y funciones de utilidad sean privadas.
- Evitar dependencias innecesarias entre clases.
- Las clases deben ser de **tamaño reducido**.
- Usar principios de diseño como **S.O.L.I.D**.



Sistemas

El diseño de sistemas limpios implica **modularidad y separación de responsabilidades**.

- Diseñar componentes pequeños y reutilizables.
- La modularidad y separación por funcionalidades permite la descentralización de la administración y la toma de decisiones.
- Usar estándares cuando añadan un valor demostrable: Los estándares facilitan la reutilización de ideas y componentes,
- Reclutan personas con experiencia.
- Usar patrones de diseño cuando sea necesario.



Sistemas

Según Kent Beck (creador de XP y TDD), un diseño es sencillo si cumple estas cuatro reglas:

1. **Ejecuta todas las pruebas:** El objetivo principal es que funcione el software como se espera y las pruebas están ahí para garantizar que así sea.
2. **Expresa la intención del programador:** El código debe ser fácil de entender.
3. **No contiene duplicados.** Aplicar el principio DRY.
4. **Minimiza el número de clases y métodos:** La última regla nos dice que todo lo que no cumpla con las tres reglas anteriores debe eliminarse.

(Las reglas están en orden de prioridad)



Concurrencia

La concurrencia introduce complejidad y debe manejarse con cuidado.

La concurrencia por lo general mejora el rendimiento.

- El diseño cambia al crear programas concurrentes
- Minimizar las regiones críticas y el uso de bloqueos.
- Evitar condiciones de carrera.
- Usar herramientas y librerías apropiadas para gestionar la concurrencia.



Código Limpio

Mejorar continuamente el código.

Refactorizar regularmente.

Seguir buenas prácticas de programación.